WeResence.

ENGINEERING TRD • V1.0

WeResence — Technical Requirements Document (TRD)

Signal Double Ratchet, Port 8901 Binary Telephony & SQLCipher Enclave Blueprint

DOCUMENT ID	**RES-WERESENCE-ENGINEERING TRD**	TARGET ARCHITECTURE	Android 8.0+ (API 26–34) / Progressive Web
LEAD ARCHITECT	**Piyush Singh** (Founder, Resence Ecosystem)	SECURITY CLASSIFICATION	**CONFIDENTIAL / PROPRIETARY SPECIFICATION**
RELEASE VERSION	v1.0.0 Production Blueprint	ECOSYSTEM DOMAIN	`resence.in` / `weresence.resence.in`

Technical Requirements Document (TRD v1.0)

Project: WeResence (Sovereign Dual-Transport Hybrid Messenger & Private AI)

Organization: Resence Ecosystem (`resence.in`)

Author: Piyush Singh (Founder & Chief Architect)

Classification: Confidential / Proprietary Engineering Specification

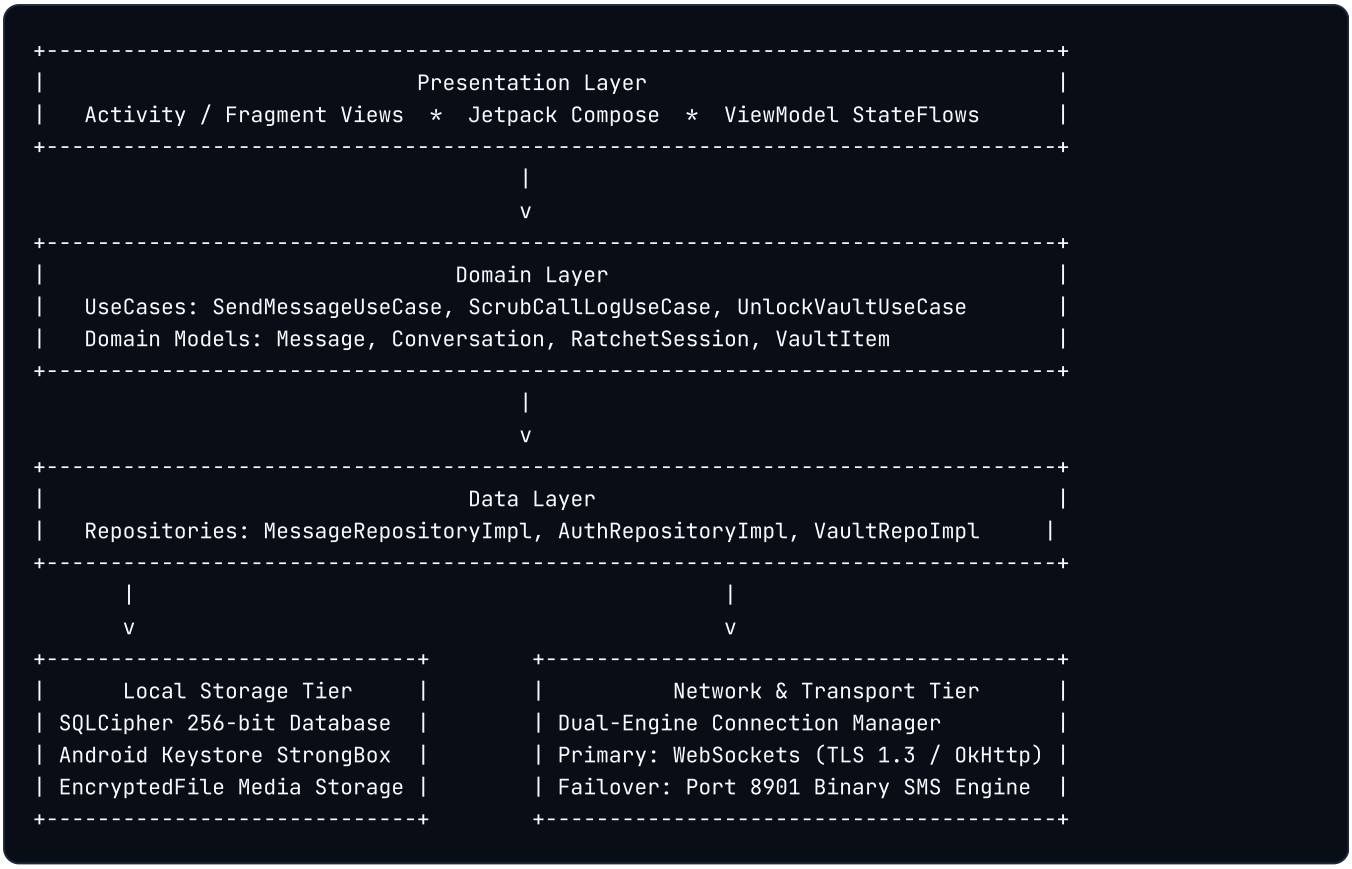
Target Stack: Kotlin 1.9+, Android SDK 26–34, Coroutines, Flow, SQLCipher, Jetpack Compose / ViewBinding

Document Status: Approved / Production Architecture Blueprint

---

1. System Architecture Overview

WeResence is structured following **Clean Architecture** and **Domain-Driven Design (DDD)** principles to enforce complete decoupling between UI presentation, business domain logic, and physical transport hardware.



---

2. Network & Transport Engineering

2.1 Dual-Engine Connection Manager

The `ConnectionManager` orchestrates automatic transport switching between high-speed IP WebSockets and low-level cellular SMS.

```
enum class TransportMode {
    INTERNET_HIGH_SPEED, // WebSockets / TLS 1.3 (Online)
    STEALTH_DATA_SMS     // Port 8901 Binary SMS (Offline Failover)
}

class ConnectionManager(private val context: Context) {
    private val connectivityManager = context.getSystemService(Context.CONNECTIVITY_SERVICE) as Connect

    var currentMode: TransportMode = TransportMode.INTERNET_HIGH_SPEED
        private set

    var onTransportModeChanged: ((TransportMode) → Unit)? = null

    // Hysteresis debounce to prevent network flapping
    private var lastSwitchTimestamp: Long = 0L
    private val DEBOUNCE_DELAY_MS = 1500L

    private val networkCallback = object : ConnectivityManager.NetworkCallback() {
        override fun onAvailable(network: Network) {
            val capabilities = connectivityManager.getNetworkCapabilities(network)
            if (capabilities?.hasCapability(NetworkCapabilities.NET_CAPABILITY_VALIDATED) == true) {
                evaluateTransportShift(TransportMode.INTERNET_HIGH_SPEED)
            }
        }

        override fun onLost(network: Network) {
            evaluateTransportShift(TransportMode.STEALTH_DATA_SMS)
        }
    }

    private fun evaluateTransportShift(newMode: TransportMode) {
        val now = SystemClock.elapsedRealtime()
        if (now - lastSwitchTimestamp > DEBOUNCE_DELAY_MS && currentMode ≠ newMode) {
            lastSwitchTimestamp = now
            currentMode = newMode
            onTransportModeChanged?.invoke(newMode)
        }
    }

    fun startMonitoring() {
        val request = NetworkRequest.Builder()
            .addCapability(NetworkCapabilities.NET_CAPABILITY_INTERNET)
            .build()
        connectivityManager.registerNetworkCallback(request, networkCallback)
    }

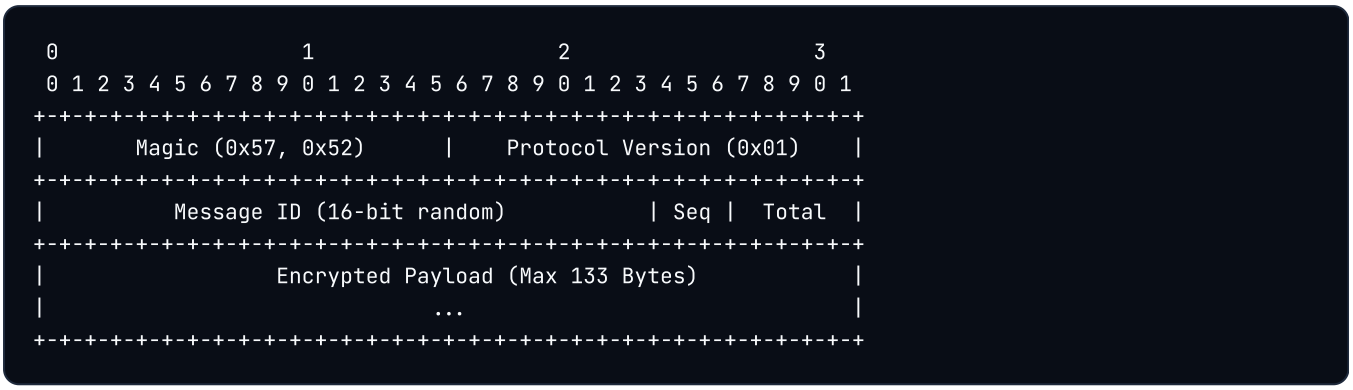
    fun stopMonitoring() {
        connectivityManager.unregisterNetworkCallback(networkCallback)
    }
}
```

2.2 Stealth Cellular Messaging: Port 8901 Binary SMS Engine

Standard mobile SMS uses textual PDU formats delivered to `content://sms/inbox`. WeResence completely bypasses this subsystem by using **port-directed data messages** (`DATA_SMS_RECEIVED_ACTION`).

PDU Binary Header & Fragmentation Protocol

Carrier SMS allows up to 140 raw bytes. WeResence allocates a **7-byte binary header**, leaving **133 bytes of encrypted payload per segment**:



Outbound Dispatch (`DataSmsSender.kt`)

```
object DataSmsSender {
    private const val DESTINATION_PORT: Short = 8901

    fun sendBinaryPayload(context: Context, destinationPhoneNumber: String, encryptedPayload: ByteArray
        val smsManager = if (Build.VERSION.SDK_INT ≥ Build.VERSION_CODES.S) {
            context.getSystemService(SmsManager::class.java)
        } else {
            @Suppress("DEPRECATION")
            SmsManager.getDefault()
        }

        val chunks = chunkPayload(encryptedPayload, 133)
        val messageId = Random.nextInt(0, 65535).toShort()
        val totalChunks = chunks.size.toByte()

        chunks.forEachIndexed { index, chunkData →
            val packet = ByteArray(7 + chunkData.size)
            // Magic bytes
            packet[0] = 0x57.toByte() // 'W'
            packet[1] = 0x52.toByte() // 'R'
            packet[2] = 0x01.toByte() // v1
            // MsgID
            packet[3] = (messageId.toInt() shr 8).toByte()
            packet[4] = (messageId.toInt() and 0xFF).toByte()
            // Chunk index & total
            packet[5] = index.toByte()
            packet[6] = totalChunks
            // Payload
            System.arraycopy(chunkData, 0, packet, 7, chunkData.size)

            smsManager.sendDataMessage(
                destinationPhoneNumber,
                null,
                DESTINATION_PORT,
                packet,
                null,
                null
            )
        }
    }

    private fun chunkPayload(data: ByteArray, chunkSize: Int): List<ByteArray> {
        val list = mutableListOf<ByteArray>()
        var offset = 0
        while (offset < data.size) {
            val length = minOf(chunkSize, data.size - offset)
            val chunk = ByteArray(length)
            System.arraycopy(data, offset, chunk, 0, length)
            list.add(chunk)
            offset += length
        }
        return list
    }
}
```

Inbound Interception (`DataSmsReceiver.kt`)

```
class DataSmsReceiver : BroadcastReceiver() {
    override fun onReceive(context: Context, intent: Intent) {
        if (intent.action == "android.intent.action.DATA_SMS_RECEIVED") {
            val uri = intent.data
            if (uri != null && uri.port == 8901) {
                val bundle = intent.extras ?: return
                val pdus = bundle.get("pdus") as? Array<*> ?: return

                for (pdu in pdus) {
                    val pduBytes = pdu as? ByteArray ?: continue
                    val format = bundle.getString("format")
                    val sms = if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.M) {
                        SmsMessage.createFromPdu(pduBytes, format)
                    } else {
                        @Suppress("DEPRECATION")
                        SmsMessage.createFromPdu(pduBytes)
                    }
                    val binaryData = sms.userData

                    if (binaryData != null && binaryData.size >= 7) {
                        if (binaryData[0] == 0x57.toByte() && binaryData[1] == 0x52.toByte()) {
                            // Process valid WeResence binary packet
                            ReassemblyEngine.ingestSegment(sms.originatingAddress ?: "", binaryData)
                            // Crucial: abort broadcast so OS SMS apps never see this packet
                            abortBroadcast()
                        }
                    }
                }
            }
        }
    }
}
```

---

3. Cryptographic Specification & Zero-Knowledge Security

3.1 Double Ratchet Protocol Architecture

WeResence implements the Signal Double Ratchet protocol, pairing a **Diffie-Hellman Ratchet** (over Curve25519) with a **Symmetric-Key KDF Ratchet** (HKDF-SHA256).

```
ARCHITECTURE FLOW DIAGRAM

sequenceDiagram
    autonumber
    participant Alice as Alice (Sender)
    participant Bob as Bob (Receiver)

    Note over Alice, Bob: X3DH Handshake completed; Master Shared Secret (SK) established
    Alice->>Alice: Step DH Ratchet (generate ephemeral keypair: EK_a)
    Alice->>Alice: DH Output = DH(EK_a, Bob_PK)
    Alice->>Alice: Root Key (RK_1), Sending Chain Key (CK_s) = HKDF(RK_0, DH Output)
    Alice->>Alice: Message Key (MK_1) = HMAC-SHA256(CK_s, 0x01)
    Alice->>Alice: Ciphertext = AES-256-GCM(MK_1, Plaintext, Nonce)
    Alice->>Bob: Transmit Ciphertext + Header (EK_a, N=1, PN=0)

    Bob->>Bob: Extract EK_a from Header
    Bob->>Bob: DH Output = DH(Bob_SK, EK_a)
    Bob->>Bob: Root Key (RK_1), Receiving Chain Key (CK_r) = HKDF(RK_0, DH Output)
    Bob->>Bob: Message Key (MK_1) = HMAC-SHA256(CK_r, 0x01)
    Bob->>Bob: Plaintext = Decrypt AES-256-GCM(MK_1, Ciphertext, Nonce)
```

3.2 Authenticated Encryption Parameters

- **Cipher**: `AES-256-GCM` (`Cipher.getInstance("AES/GCM/NoPadding")`)
- **Key Length**: 256 bits (32 bytes)
- **Initialization Vector (IV / Nonce)**: 96 bits (12 bytes) cryptographically random per message via `SecureRandom`
- **Authentication Tag**: 128 bits (16 bytes)

---

4. Android Hardware Security & Encrypted Storage

4.1 Hardware Keystore Provider (`AndroidKeyStore`)

Root encryption keys are anchored in the device's hardware security module (**StrongBox Keymaster** or **TEE**):

```
object KeyStoreManager {
    private const val ANDROID_KEYSTORE = "AndroidKeyStore"
    private const val KEY_ALIAS = "WeResenceMasterKey"

    fun getOrCreateMasterKey(): SecretKey {
        val keyStore = KeyStore.getInstance(ANDROID_KEYSTORE).apply { load(null) }
        if (keyStore.containsAlias(KEY_ALIAS)) {
            val entry = keyStore.getEntry(KEY_ALIAS, null) as KeyStore.SecretKeyEntry
            return entry.secretKey
        }

        val keyGenerator = KeyGenerator.getInstance(
            KeyProperties.KEY_ALGORITHM_AES,
            ANDROID_KEYSTORE
        )

        val builder = KeyGenParameterSpec.Builder(
            KEY_ALIAS,
            KeyProperties.PURPOSE_ENCRYPT or KeyProperties.PURPOSE_DECRYPT
        )
            .setBlockModes(KeyProperties.BLOCK_MODE_GCM)
            .setEncryptionPaddings(KeyProperties.ENCRYPTION_PADDING_NONE)
            .setKeySize(256)
            .setUserAuthenticationRequired(false) // Set true for biometrics prompt

        if (Build.VERSION.SDK_INT ≥ Build.VERSION_CODES.P) {
            // Prefer StrongBox Hardware Security Enclave if available
            builder.setIsStrongBoxBacked(true)
        }

        keyGenerator.init(builder.build())
        return keyGenerator.generateKey()
    }
}
```

4.2 SQLCipher 256-bit Database Architecture

The local database runs on **SQLCipher** over SQLite, encrypted with a 256-bit passphrase derived via **PBKDF2 with 64,000 iterations**.

```
-- Room Database Schema (Encrypted SQLCipher Tables)

CREATE TABLE IF NOT EXISTS users (
    phone_number TEXT PRIMARY KEY NOT NULL,
    resence_id TEXT UNIQUE NOT NULL,
    display_name TEXT NOT NULL,
    public_identity_key BLOB NOT NULL,
    avatar_url TEXT,
    bio TEXT,
    is_verified INTEGER NOT NULL DEFAULT 0,
    created_at INTEGER NOT NULL
);

CREATE TABLE IF NOT EXISTS chats (
    chat_id TEXT PRIMARY KEY NOT NULL,
    peer_phone TEXT NOT NULL,
    is_community INTEGER NOT NULL DEFAULT 0,
    community_name TEXT,
    last_message_preview TEXT,
    last_message_timestamp INTEGER NOT NULL,
    unread_count INTEGER NOT NULL DEFAULT 0,
    current_transport TEXT NOT NULL DEFAULT 'INTERNET_HIGH_SPEED',
    FOREIGN KEY(peer_phone) REFERENCES users(phone_number) ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS messages (
    message_id TEXT PRIMARY KEY NOT NULL,
    chat_id TEXT NOT NULL,
    sender_phone TEXT NOT NULL,
    recipient_phone TEXT NOT NULL,
    content_ciphertext BLOB NOT NULL,
    content_plaintext TEXT, -- Populated strictly in RAM, cleared on memory release
    transport_used TEXT NOT NULL, -- 'INTERNET_HIGH_SPEED' or 'STEALTH_DATA_SMS'
    delivery_status TEXT NOT NULL, -- 'PENDING', 'SENT', 'DELIVERED', 'READ'
    timestamp INTEGER NOT NULL,
    is_burned INTEGER NOT NULL DEFAULT 0,
    FOREIGN KEY(chat_id) REFERENCES chats(chat_id) ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS mutual_vault_items (
    item_id TEXT PRIMARY KEY NOT NULL,
    chat_id TEXT NOT NULL,
    uploader_phone TEXT NOT NULL,
    media_path_encrypted TEXT NOT NULL,
    media_mime_type TEXT NOT NULL,
    file_size_bytes INTEGER NOT NULL,
    uploader_signature BLOB NOT NULL,
    created_at INTEGER NOT NULL,
    FOREIGN KEY(chat_id) REFERENCES chats(chat_id) ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS ratchet_states (
    peer_phone TEXT PRIMARY KEY NOT NULL,
    root_key BLOB NOT NULL,
    sending_chain_key BLOB NOT NULL,
    receiving_chain_key BLOB NOT NULL,
    dh_local_private_key BLOB NOT NULL,
    dh_local_public_key BLOB NOT NULL,
    dh_remote_public_key BLOB NOT NULL,
    previous_counter INTEGER NOT NULL DEFAULT 0,
```

```
    send_counter INTEGER NOT NULL DEFAULT 0,
    receive_counter INTEGER NOT NULL DEFAULT 0
);
```

---

5. Privacy Engineering & Android OS Hooking

5.1 Hardware Surface Protection (`FLAG_SECURE`)

To eliminate the threat of screenshotting, screen mirrors, HDMI captures, and background task preview caching, every Activity enforces `FLAG_SECURE` in its lifecycle initialization:

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    window.setFlags(
        WindowManager.LayoutParams.FLAG_SECURE,
        WindowManager.LayoutParams.FLAG_SECURE
    )
}
```

5.2 Native Call Log Scrubbing Engine (`CallLogManager.kt`)

Android voice calling can inadvertently write call records to the system content provider. WeResence registers a `ContentObserver` on `CallLog.Calls.CONTENT_URI` to immediately purge any matched session record:

```
class CallLogScrubber(
    private val context: Context,
    private val peerNumber: String
) : ContentObserver(Handler(Looper.getMainLooper())) {

    override fun onChange(selfChange: Boolean) {
        super.onChange(selfChange)
        purgeCallEntry()
    }

    private fun purgeCallEntry() {
        CoroutineScope(Dispatchers.IO).launch {
            try {
                val resolver = context.contentResolver
                val where = "${CallLog.Calls.NUMBER} = ?"
                val rowsDeleted = resolver.delete(
                    CallLog.Calls.CONTENT_URI,
                    where,
                    arrayOf(peerNumber)
                )
                Log.d("CallLogScrubber", "Purged $rowsDeleted native call log entry.")
            } catch (e: SecurityException) {
                Log.w("CallLogScrubber", "Permission not granted: ${e.message}")
            }
        }
    }
}
```

5.3 Decoy Ghost PIN Engine

- When unlocking WeResence, the user inputs a 4-digit PIN.
- The input is hashed via PBKDF2:
- If `hash == MASTER_HASH` (`1234`), initialize the production SQLCipher instance `weresence_master.db`.
- If `hash == DECOY_HASH` (`0000`), initialize an isolated ephemeral decoy database `weresence_decoy.db` pre-populated with benign generic chats and zero traces of true user data.
- Under duress, providing `0000` unlocks a realistic decoy workspace without compromising master keys.

---

6. Resence AI Neural Gateway (Zero-Key Client Proxy)

6.1 Architectural Decoupling

To eliminate the vulnerability of reverse-engineering APK files to extract API keys:
1. **The Android client has 0 developer API keys.**
2. Client queries connect to the **Resence AI Proxy Gateway** (`https://api.resence.in/v1/ai/chat`) authenticated via a short-lived device session token.
3. The server gateway holds pooled developer API keys in secure `.env` variables and implements load-balancing across models.

6.2 Resence AI Service Implementation (`ResenceAiService.kt`)

```
class ResenceAiService(
    private val gatewayUrl: String = "https://api.resence.in/v1/ai/chat",
    private val userSessionToken: String = ""
) {
    suspend fun processQuery(request: ResenceAiRequest, isOnline: Boolean): ResenceAiResponse = withCor
        // Enforce offline quota freeze
        if (!isOnline) {
            return@withContext ResenceAiResponse(
                replyText = "Resence AI requires active data to think. Offline SMS is reserved exclusiv
            )
        }

        val url = URL(gatewayUrl)
        val conn = (url.openConnection() as HttpURLConnection).apply {
            requestMethod = "POST"
            setRequestProperty("Content-Type", "application/json")
            setRequestProperty("Authorization", "Bearer $userSessionToken")
            connectTimeout = 8000
            readTimeout = 15000
            doOutput = true
        }

        val payload = JSONObject().apply {
            put("prompt", request.prompt)
            put("hasImage", request.base64Image ≠ null)
            request.base64Image?.let { put("image", it) }
        }

        OutputStreamWriter(conn.outputStream).use { it.write(payload.toString()) }

        val responseCode = conn.responseCode
        if (responseCode == 200) {
            val text = BufferedReader(InputStreamReader(conn.inputStream)).use { it.readText() }
            val json = JSONObject(text)
            return@withContext ResenceAiResponse(replyText = json.optString("reply"))
        } else {
            return@withContext ResenceAiResponse(replyText = "AI Gateway synchronized standby.")
        }
    }
}
```

7. STRIDE Threat Model & Security Controls

Threat Category	Threat Description	WeResence Mitigation Control
Spoofing	Attacker impersonates a peer phone number over cellular SMS	Every message payload is signed with the peer's Curve25519 Identity Key. Forged SMS packets fail HMAC-SHA256 authentication and are silently discarded.
Tampering	Man-in-the-middle alters ciphertext during WebSocket or SMS transit	AES-256-GCM produces a 128-bit authentication tag. Any modified byte fails cryptographic validation with `AEADBadTagException`.
Repudiation	User claims they did not send a specific Mutual Vault upload	Uploaded vault media contains an immutable Ed25519 digital signature of the sender's public key.
Information Disclosure	Telecom operator snoops cellular SMS packets	Port 8901 binary payloads are AES-256-GCM encrypted byte arrays. To the telecom carrier, they appear as unstructured high-entropy noise.
Denial of Service	Adversary spams Port 8901 with random binary garbage	High-priority BroadcastReceiver checks magic header (`0x57, 0x52`). If magic fails, the packet is instantly discarded without invoking the crypto engine.
Elevation of Privilege	Recipient tries to delete files uploaded by partner in Mutual Vault	Deletion requests verify the cryptographic signature of the requester against the uploader's public key. Non-uploaders cannot invoke delete APIs.

8. Build Configuration & ProGuard/R8 Rules

```
# WeResence R8 Obfuscation & Security Rules

# Strip all debug logs in production release
-assumenosideeffects class android.util.Log {
    public static boolean isLoggable(java.lang.String, int);
    public static int v(...);
    public static int d(...);
    public static int i(...);
    public static int w(...);
}

# Preserve cryptographic primitives and SQLCipher JNI bindings
-keep class net.sqlcipher.** { *; }
-keep class net.sqlcipher.database.** { *; }
-keep class org.whispersystems.curve25519.** { *; }

# Obfuscate all domain and UI classes
-repackageclasses 'com.resence.weresence.internal'
-allowaccessmodification
```